Stage 5 Periodic Ring Infrastructure Specification

API Hygiene, Source-Safe Identity, Shared Periodic Arithmetic, and Cycle Parametrization

mdstats

2026-07-18

Contents

1 Purpose

This specification freezes the lightweight Stage-5 infrastructure after the three consumer prototypes have been implemented and tested:

1. exact translated primitive-ring placement;

2. exact automorphism-induced ordered ring-occurrence mapping; and
3. exact finite GF(2) cancellation over translated smaller-ring support.

The cleanup deliberately extracts only operations that were duplicated by real consumers. It does **not** implement `PeriodicNetView`, automatic symmetry discovery, strong-ring domain enumeration, embedded faces, or natural tiling.

Runtime/API version:

`mdstats 0.19.13a0`

2 Motive

The prototypes validated the underlying representation, but exposed three API risks:

- dense edge indices and ring keys could be interpreted against the wrong source topology;
- periodic shift/matrix arithmetic had been duplicated across modules; and
- automorphism code depended on a private primitive-ring canonicalization helper.

The cleanup therefore establishes four invariants:

1. cross-module periodic identities are source-bound by topology digest;
2. exact physical edge instances use stable `FrameworkEdgeKey`, not bare dense edge indices;
3. physical placement is distinct from cycle parametrization; and
4. shared periodic arithmetic is private and representation-neutral.

3 Algorithmic provenance

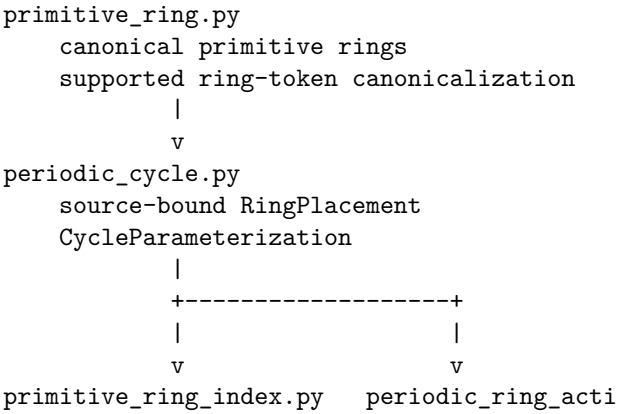
The periodic quotient-edge representation follows the vector/quotient-graph method of Chung, Hahn, and Klee [1] and the later quotient-graph discussion by Klee [2]. The automorphism representation used by the Stage-5 action prototype follows the exact periodic-net viewpoint of Delgado-Friedrichs and O'Keeffe [3]. The finite ring-cancellation concept follows the strong-ring literature of Goetzke and Klein [4] and Yuan and Cormack [5].

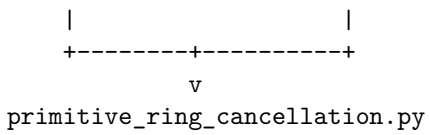
The following are `mdstats` design decisions rather than copied algorithms:

- source-bound `RingPlacement` and `LiftedEdgeInstanceRef` records;
- separation of `CycleParameterization` from physical placement;
- stable-key index lookup and exact ordered support accessors;
- top-level export restriction of advanced Stage-5 infrastructure; and
- the exact division between private periodic arithmetic and scientific modules.

Implementation comments adjacent to quotient-edge anchor arithmetic retain references [1,2]. Automorphism code retains [1,3]. Ring cancellation retains [4,5].

4 Module boundary





```
_private support:
    _periodic_graph.py
```

`PrimitiveRingCatalog` remains the sole scientific ring catalog.

5 Shared private periodic arithmetic

Module:

`mdstats/analysis/_periodic_graph.py`

This module is private. It may contain only exact, representation-neutral integer-lattice operations:

```
coerce_lattice_shift(...)
coerce_int_matrix3(...)
add_shift(...)
subtract_shift(...)
negate_shift(...)
determinant3(...)
matvec_shift(...)
physical_edge_anchor(...)
```

It must not contain:

- ring canonicalization;
- graph search;
- symmetry discovery;
- ring catalogs;
- public chain algebra; or
- natural-tiling policy.

For a quotient edge

$$e = (i, j, \Delta), \qquad \Delta \in \mathbb{Z}^3,$$

anchored at image $\mathbf{a}$, the physical edge is

$$(i, \mathbf{a}) \to (j, \mathbf{a} + \Delta).$$

If traversed in reverse from lifted source image $\mathbf{s}$, the canonical endpoint anchor is

$$\mathbf{a} = \mathbf{s} - \Delta.$$

6 Supported primitive-ring canonicalization

Module:

`mdstats/analysis/primitive_ring.py`

API:

```
def canonicalize_primitive_ring_tokens(
    tokens: tuple[PrimitiveRingEdgeToken, ...],
) -> tuple[PrimitiveRingEdgeToken, ...]:
    ...
```

The returned sequence is the lexicographically minimum cyclic rotation over both forward and completely reversed orientations.

Constraints:

- tokens must describe one intended cyclic boundary;
- this function canonicalizes identity only; it does not validate graph closure;
- `PrimitiveRingKey` remains responsible for requiring canonical token order.

This function is package-supported through `mdstats.analysis`; it is not exported from the package root.

7 periodic_cycle.py

7.1 RingPlacement

```
@dataclass(frozen=True, order=True, slots=True)
class RingPlacement:
    topology_graph_digest: str
    ring_key: PrimitiveRingKey
    image_shift: tuple[int, int, int]
```

Meaning:

- `topology_graph_digest` binds the source-local ring key to one framework graph;
- `ring_key` identifies the canonical primitive-ring translation orbit; and
- `image_shift` translates that canonical representative.

Persistent identity is therefore

$$(\text{topology graph digest}, \texttt{PrimitiveRingKey}, \mathbf{t}).$$

Constraints:

- digest is nonempty;
- `ring_key` is a `PrimitiveRingKey`;
- shift contains exactly three integers.

A `RingPlacement` must be rejected if supplied to an index or consumer with a different topology digest.

7.2 CycleParameterization

```
@dataclass(frozen=True, order=True, slots=True)
class CycleParameterization:
    start_vertex_index: int = 0
    orientation: Literal[-1, 1] = 1

    def vertex_permutation(self, size: int) -> tuple[int, ...]: ...
    def step_permutation(self, size: int) -> tuple[int, ...]: ...
```

For cycle size n, start position c, and orientation $\epsilon \in \{-1, +1\}$,

$$p_V(k) = c + \epsilon k \pmod{n}.$$

Edge-step positions are

$$p_E(k) = \begin{cases} c + k \pmod{n}, & \epsilon = +1, \\ c - k - 1 \pmod{n}, & \epsilon = -1. \end{cases}$$

This object changes only boundary parametrization. It does not create a new physical placement.

8 primitive_ring_index.py

8.1 LiftedEdgeInstanceRef

```python
@dataclass(frozen=True, order=True, slots=True)
class LiftedEdgeInstanceRef:
    topology_graph_digest: str
    edge_key: FrameworkEdgeKey
    anchor_shift: tuple[int, int, int]
```

This is the exact identity of one physical translated framework-edge instance.

The dense source edge index is intentionally absent from the cross-module record. Dense indices remain transient acceleration handles inside `PrimitiveRingIndex`.

Constraints:

- digest is nonempty;
- `edge_key` is a complete `FrameworkEdgeKey`;
- anchor contains exactly three integers.

8.2 PrimitiveRingIndex

```python
@dataclass(frozen=True, slots=True)
class PrimitiveRingIndex:
    catalog: PrimitiveRingCatalog
    ring_keys: tuple[PrimitiveRingKey, ...]
    # edge occurrence storage is private

    @property
    def edge_count(self) -> int: ...

    @property
    def occurrence_count(self) -> int: ...

    def ring_id_for_key(self, key: PrimitiveRingKey) -> int: ...
    def ring_for_key(self, key: PrimitiveRingKey) -> PrimitiveRing: ...
    def edge_index_for_key(self, key: FrameworkEdgeKey) -> int: ...
    def edge_key_for_index(self, edge_index: int) -> FrameworkEdgeKey: ...

    def canonical_edge_instance(
        self,
        key: PrimitiveRingKey,
        step_index: int,
    ) -> LiftedEdgeInstanceRef: ...

    def canonical_edge_instances(
        self,
        key: PrimitiveRingKey,
    ) -> tuple[LiftedEdgeInstanceRef, ...]: ...

    def translated_edge_instances(
```

```
        self,
        placement: RingPlacement,
    ) -> tuple[LiftedEdgeInstanceRef, ...]: ...
```

`canonical_edge_instances()` returns the ordered physical-edge support of the stored canonical representative. Translation is then exactly

$$(e_k, \mathbf{a}_k) \mapsto (e_k, \mathbf{a}_k + \mathbf{t}).$$

The private occurrence buckets are implementation detail and are not exported as a public record type.

8.3 Placement query

```
def ring_placements_covering_edge(
    index: PrimitiveRingIndex,
    edge_instance: LiftedEdgeInstanceRef,
) -> tuple[RingEdgePlacement, ...]:
    ...
```

For canonical occurrence anchor $\mathbf{a}_k$ and requested physical anchor $\mathbf{a}$,

$$\mathbf{t} = \mathbf{a} - \mathbf{a}_k.$$

The function must reject a mismatched topology digest before resolving the edge key.

9 Automorphism occurrence API cleanup

`ValidatedPeriodicAutomorphism` remains a Stage-5 validation/application record; it is **not yet the final `PeriodicNetView`-owned symmetry object**.

`RingOccurrenceMap` now stores authoritative position permutations:

```
@dataclass(frozen=True, slots=True)
class RingOccurrenceMap:
    topology_graph_digest: str
    source_placement: RingPlacement
    target_placement: RingPlacement
    source_vertex_position_to_target_position: tuple[int, ...]
    source_step_position_to_target_position: tuple[int, ...]
    parameterization: CycleParameterization
```

`orientation` and `start_vertex_index` are derived convenience properties of `parameterization`.

The target ring key is built through `canonicalize_primitive_ring_tokens()`; cross-module code must not import a private underscore helper from `primitive_ring.py`.

10 Finite cancellation API interaction

`ring_placement_support()` now delegates exact translated support generation to `PrimitiveRingIndex.translated_edge_ins`
This removes duplicate support reconstruction logic.

The finite solver remains

$$[T] \in \operatorname{span}_{\mathrm{GF}(2)}\{[R_1], \ldots, [R_N]\}.$$

`NOT_IN_SUPPLIED_SPAN` means only that no decomposition exists in the explicitly supplied finite candidate set. It is not a strong-ring theorem.

The future mathematical strength domain should not impose an independent `max_component_count`: over a finite GF(2) candidate set, each basis candidate appears with coefficient 0 or 1, so existence is ordinary span membership. Resource limits remain separate from scientific domain definition.

11 Export policy

Advanced Stage-5 infrastructure is exported from:

`mdstats.analysis`

but not re-exported from:

`mdstats`

This keeps the package-root API focused on stable scientific/high-level entry points while Stage-5 symmetry and strength contracts are still alpha-stage.

12 Input and source constraints

All Stage-5 consumers must enforce:

1. matching `topology_graph_digest` for source-bound placements and edge refs;
2. stable ring-key membership in the owning `PrimitiveRingCatalog`;
3. stable edge-key membership in the owning source framework graph;
4. exact integer lattice shifts;
5. no promotion of catalog completeness beyond the source primitive-ring result;
6. no interpretation of dense ring/edge IDs outside the owning transient index.

13 Edge cases

- **Structurally identical but unrelated topology:** identical-looking edge/ring keys are insufficient; digest mismatch must reject the operation.
- **Parallel framework edges:** complete `FrameworkEdgeKey` identity keeps them distinct.
- **Periodic self-images:** image shift is part of physical placement and must not be discarded.
- **Reversed cycle traversal:** handled by `CycleParameterization`; it is not a distinct physical ring placement.
- **Bound refinement:** dense IDs may change when the primitive-ring bound grows; stable keys plus source digest remain authoritative.
- **Incomplete/truncated source catalog:** every Stage-5 operation is exact only relative to represented rings; no downstream negative theorem may exceed source completeness.
- **Future net views:** symmetry operations must later be bound to `PeriodicNetView.digest`; `ValidatedPeriodicAutomorphism` remains provisional.

14 Tests and acceptance gate

The cleanup gate requires:

- direct P1-P3 tests;
- cross-topology rejection for ring placements and physical edge instances;
- cycle parametrization forward/reverse permutation tests;
- exact canonical-to-translated support tests;
- package-root export regression;
- Stage-4R/framework periodic regression; and
- complete package test suite.

Accepted result for `0.19.13a0`:

```
586 passed, 28 warnings
```

The suite was executed in three nonoverlapping file groups because one monolithic run exceeded the execution-window timeout; every collected test file was covered exactly once across those groups.

15 Deferred work

The next scientific stage is `PeriodicNetView`:

```
FrameworkTopology
    -> PeriodicNetView(signature policy)
    -> validated/discovered net automorphisms
    -> ring occurrence action
```

Automatic symmetry discovery, strength-domain enumeration, geometric face construction, periodic spatial broad phase, and natural tiling remain downstream.

16 References

1. Chung, S. J., Hahn, Th., and Klee, W. E. (1984). *Nomenclature and generation of three-periodic nets: the vector method*. Acta Crystallographica A 40, 42-50. DOI: 10.1107/S0108767384000088.
2. Klee, W. E. (2004). *Crystallographic nets and their quotient graphs*. Cryst. Res. Technol. 39, 959-968. DOI: 10.1002/crat.200410281.
3. Delgado-Friedrichs, O., and O'Keeffe, M. (2003). *Identification of and symmetry computation for crystal nets*. Acta Crystallographica A 59, 351-360. DOI: 10.1107/S0108767303012017.
4. Goetzke, K., and Klein, H.-J. (1991). *Properties and efficient algorithmic determination of different classes of rings in finite and infinite polyhedral networks*. J. Non-Cryst. Solids 127, 215-220. DOI: 10.1016/0022-3093(91)90145-V.
5. Yuan, X., and Cormack, A. N. (2002). *Efficient algorithm for primitive ring statistics in topological networks*. Comput. Mater. Sci. 24, 343-360. DOI: 10.1016/S0927-0256(01)00256-7.